FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	**OMBROG, Xavier Michael Emmanuel, N.**
Section:	**AN43**
Professor:	
Date:	

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

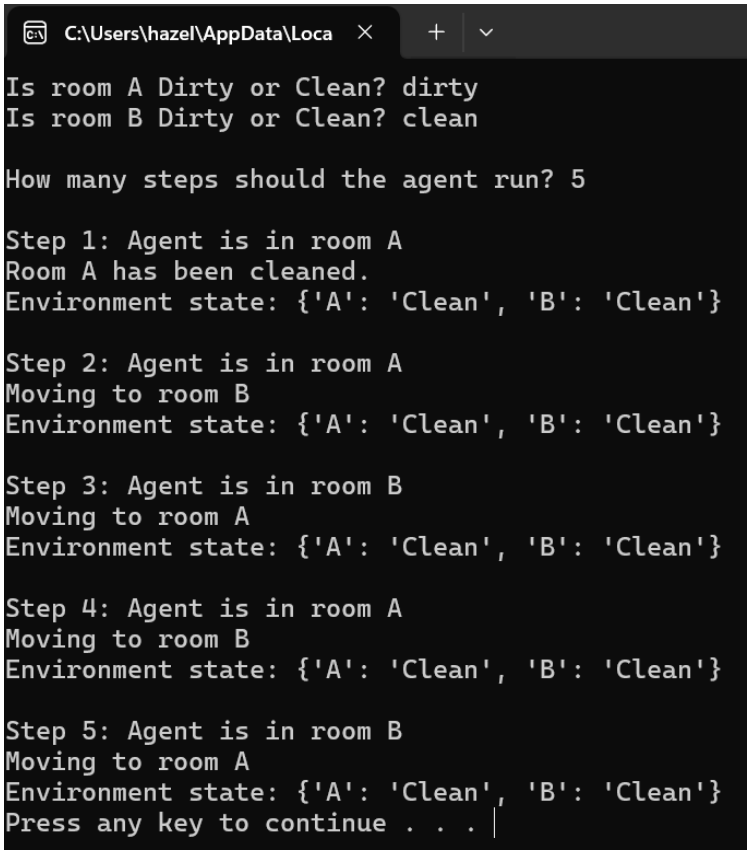
Pre-Lab Preparation

1. Review the Python concepts:
    - Classes and objects
    - If-Else statements
    - Loops (for loop)
    - Functions / methods
2. Understand the environment:
    - Two rooms: A and B
    - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
    - Rule 1: If the current room is dirty, clean it.

- Rule 2: If the current room is clean, move to the other room.

3. Implement a **move method** to switch rooms.

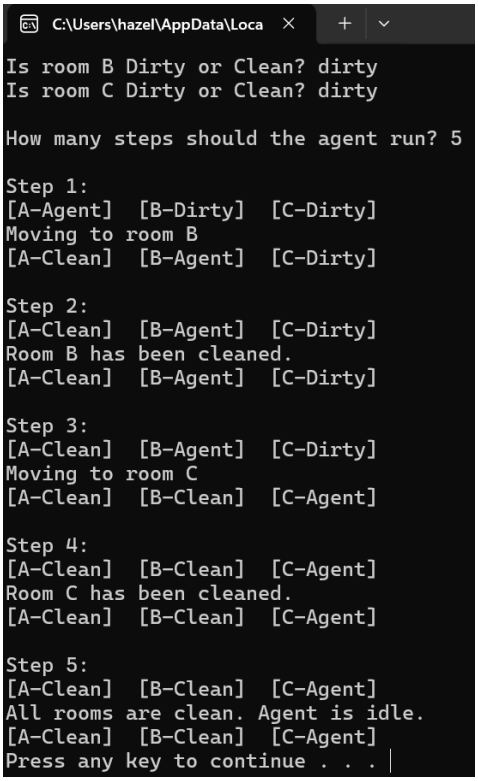
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
   - The agent's current location
   - The action taken (cleaned/moved)
   - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

My program is in TA2_rule_based_agent.py. It opens with a small menu so the main tasks and the bonus task can both run from one file. Option 1 covers Tasks 1 to 3. The rooms are stored in a dictionary, the user sets each room's starting state, and is_dirty() and clean_room() handle Task 1. The VacuumAgent class

has the perceive_and_act() method with Rules 1 and 2 and a move() method that switches rooms. Option 2 is the bonus task. MultiRoomVacuumAgent adds room C, moves to a randomly chosen dirty room and goes idle once every room is clean, and show_grid() draws the rooms as a text-based grid. Every input is checked, so an invalid answer just gets asked again, and each run ends with a short summary of the actions taken.

TA2_rule_based_agent.py

```
"""
CS0065 - Intelligent Systems
Technical Assessment 2: Rule-Based Agent (Vacuum Cleaner)

Task 1 - Set up the environment: the rooms are stored in a dictionary, the user
         sets the initial state of each room, and two helper functions check
         whether a room is dirty and clean a room.
Task 2 - Rule-based agent: the VacuumAgent class has a perceive_and_act()
         method that follows two rules and a move() method that switches rooms.
           Rule 1: If the current room is dirty, clean it.
           Rule 2: If the current room is clean, move to the other room.
Task 3 - Simulation: the user enters the number of steps, and after every step
         the program prints the agent's location, the action taken and the
         current state of the environment.
Bonus  - Three rooms (A, B, C): the agent randomly chooses which dirty room to
         clean next, stays idle once every room is clean, and its movement is
         shown on a text-based grid.
"""

import random

CLEAN = "Clean"
DIRTY = "Dirty"


# ============================================================================
# TASK 1: SET UP THE ENVIRONMENT
# ============================================================================
def ask_room_state(room):
    """Ask the user whether a room starts Dirty or Clean (re-asks on bad input)."""
    while True:
        answer = input(f"Is room {room} Dirty or Clean? ").strip().lower()
        if answer in ("dirty", "d"):
            return DIRTY
        if answer in ("clean", "c"):
            return CLEAN
        print("   Invalid input. Please type Dirty or Clean.")


def create_environment(rooms):
    """Build the environment dictionary, e.g. {'A': 'Dirty', 'B': 'Clean'}."""
    environment = {}
    for room in rooms:
        environment[room] = ask_room_state(room)
    return environment


def is_dirty(environment, room):
    """Return True if the given room is dirty."""
    return environment[room] == DIRTY


def clean_room(environment, room):
    """Clean the given room by updating its state in the environment."""
    environment[room] = CLEAN


def ask_steps():
    """Ask how many steps the simulation should run (must be 1 or more)."""
    while True:
```

```
        answer = input("How many steps should the agent run? ").strip()
        try:
            steps = int(answer)
        except ValueError:
            steps = 0
        if steps > 0:
            return steps
        print("   Invalid input. Please enter a whole number greater than 0.")


# ============================================================================
# TASK 2: IMPLEMENT THE RULE-BASED AGENT
# ============================================================================
class VacuumAgent:
    """Simple rule-based vacuum cleaner for two rooms (A and B)."""

    def __init__(self, start_room="A"):
        self.location = start_room      # the room the agent is currently in

    def move(self):
        """Switch to the other room (A -> B or B -> A)."""
        if self.location == "A":
            self.location = "B"
        else:
            self.location = "A"

    def perceive_and_act(self, environment):
        """Perceive the current room, apply the rules, and return the action taken."""
        if is_dirty(environment, self.location):
            # Rule 1: If the current room is dirty, clean it.
            clean_room(environment, self.location)
            return "cleaned", f"Room {self.location} has been cleaned."
        # Rule 2: If the current room is clean, move to the other room.
        self.move()
        return "moved", f"Moving to room {self.location}"


# ============================================================================
# TASK 3: RUN THE SIMULATION (TWO ROOMS)
# ============================================================================
def run_two_room_simulation():
    # INPUT: initial state of each room and number of steps
    environment = create_environment(["A", "B"])
    print()
    steps = ask_steps()

    # PROCESS: the agent starts in room A and applies its rules once per step
    agent = VacuumAgent(start_room="A")
    counts = {"cleaned": 0, "moved": 0}
    for step in range(1, steps + 1):
        print()
        print(f"Step {step}: Agent is in room {agent.location}")
        action, message = agent.perceive_and_act(environment)
        counts[action] += 1

        # OUTPUT after each step: action taken and current state of the environment
        print(message)
        print(f"Environment state: {environment}")

    # OUTPUT: summary of the whole run
    print()
    print(f"Simulation finished after {steps} step(s): "
          f"{counts['cleaned']} clean action(s), {counts['moved']} move(s).")
    print(f"Final location: room {agent.location} | Final state: {environment}")


# ============================================================================
# BONUS TASK: THREE ROOMS, RANDOM CHOICE OF DIRTY ROOM, TEXT-BASED GRID
# ============================================================================
```

```
class MultiRoomVacuumAgent(VacuumAgent):
    """Rule-based vacuum cleaner that works with any number of rooms."""

    def move_to(self, room):
        """Move directly to the chosen room."""
        self.location = room

    def perceive_and_act(self, environment):
        """Apply the updated rules for three (or more) rooms."""
        if is_dirty(environment, self.location):
            # Rule 1: If the current room is dirty, clean it.
            clean_room(environment, self.location)
            return "cleaned", f"Room {self.location} has been cleaned."

        dirty_rooms = [room for room in environment if is_dirty(environment, room)]
        if dirty_rooms:
            # Rule 2: If the current room is clean, move to a randomly chosen dirty room.
            self.move_to(random.choice(dirty_rooms))
            return "moved", f"Moving to room {self.location}"

        # Rule 3: If every room is clean, stay where you are.
        return "idle", "All rooms are clean. Agent is idle."


def show_grid(environment, agent_location):
    """Text-based grid: the agent's room shows 'Agent', other rooms show their state."""
    cells = []
    for room in environment:
        label = "Agent" if room == agent_location else environment[room]
        cells.append(f"[{room}-{label}]")
    print("  ".join(cells))


def run_three_room_simulation():
    # INPUT: initial state of rooms A, B and C and number of steps
    environment = create_environment(["A", "B", "C"])
    print()
    steps = ask_steps()

    # PROCESS: the agent starts in room A and applies its rules once per step
    agent = MultiRoomVacuumAgent(start_room="A")
    counts = {"cleaned": 0, "moved": 0, "idle": 0}
    for step in range(1, steps + 1):
        print()
        print(f"Step {step}:")
        show_grid(environment, agent.location)          # before the action
        action, message = agent.perceive_and_act(environment)
        counts[action] += 1

        # OUTPUT after each step: action taken and the updated grid
        print(message)
        show_grid(environment, agent.location)          # after the action

    # OUTPUT: summary of the whole run
    print()
    print(f"Simulation finished after {steps} step(s): "
          f"{counts['cleaned']} clean action(s), {counts['moved']} move(s), "
          f"{counts['idle']} idle step(s).")
    print(f"Final location: room {agent.location} | Final state: {environment}")


# ============================================================================
# MAIN PROGRAM
# ============================================================================
def main():
    print("=" * 55)
    print("RULE-BASED AGENT: VACUUM CLEANER SIMULATION")
    print("=" * 55)
    while True:
```

```
        print("[1] Two rooms: A and B (Tasks 1-3)")
        print("[2] Three rooms: A, B and C with text-based grid (Bonus Task)")
        print("[3] Exit")
        choice = input("Choose an option (1-3): ").strip()
        print()
        if choice == "1":
            run_two_room_simulation()
        elif choice == "2":
            run_three_room_simulation()
        elif choice == "3":
            print("Program ended.")
            break
        else:
            print("   Invalid choice. Please enter 1, 2 or 3.")
            continue
        print("\n" + "-" * 55)


if __name__ == "__main__":
    main()
```

Sample Run 1: Two Rooms (Tasks 1 to 3)

Inputs: option 1, room A dirty, room B clean, 5 steps (the same inputs as the sample output).

Console Output - Sample Run 1

```
============================================================
RULE-BASED AGENT: VACUUM CLEANER SIMULATION
============================================================
[1] Two rooms: A and B (Tasks 1-3)
[2] Three rooms: A, B and C with text-based grid (Bonus Task)
[3] Exit
Choose an option (1-3): 1

Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Simulation finished after 5 step(s): 1 clean action(s), 4 move(s).
Final location: room A | Final state: {'A': 'Clean', 'B': 'Clean'}

-------------------------------------------------------
[1] Two rooms: A and B (Tasks 1-3)
[2] Three rooms: A, B and C with text-based grid (Bonus Task)
[3] Exit
Choose an option (1-3): 3

Program ended.
```

Run 1 observation. The steps match the sample output line for line. Room A was dirty, so Rule 1 fired and the agent cleaned it on step 1. After that both rooms were clean, so Rule 2 fired on every step and the agent kept moving between A and B (1 clean action, 4 moves).

Sample Run 2: Both Rooms Dirty, with Input Validation

Inputs: option 1, an invalid room state (messy), room A dirty, room B dirty, an invalid step count (0), then 6 steps.

Console Output - Sample Run 2

```
======================================================
RULE-BASED AGENT: VACUUM CLEANER SIMULATION
======================================================
[1] Two rooms: A and B (Tasks 1-3)
[2] Three rooms: A, B and C with text-based grid (Bonus Task)
[3] Exit
Choose an option (1-3): 1

Is room A Dirty or Clean? messy
   Invalid input. Please type Dirty or Clean.
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? dirty

How many steps should the agent run? 0
   Invalid input. Please enter a whole number greater than 0.
How many steps should the agent run? 6

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Dirty'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Dirty'}

Step 3: Agent is in room B
Room B has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 6: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Simulation finished after 6 step(s): 2 clean action(s), 4 move(s).
Final location: room A | Final state: {'A': 'Clean', 'B': 'Clean'}

------------------------------------------------------
[1] Two rooms: A and B (Tasks 1-3)
[2] Three rooms: A, B and C with text-based grid (Bonus Task)
[3] Exit
Choose an option (1-3): 3

Program ended.
```

Run 2 observation. The program rejected "messy" and 0 and asked again. With both rooms dirty, the agent cleaned A, moved to B and cleaned B, then spent the last three steps moving back and forth (2 clean actions, 4 moves).

Sample Run 3: Both Rooms Already Clean (supports Question 3)

Inputs: option 1, room A clean, room B clean, 4 steps.

Console Output - Sample Run 3

```
======================================================
RULE-BASED AGENT: VACUUM CLEANER SIMULATION
======================================================
[1] Two rooms: A and B (Tasks 1-3)
[2] Three rooms: A, B and C with text-based grid (Bonus Task)
[3] Exit
Choose an option (1-3): 1

Is room A Dirty or Clean? clean
Is room B Dirty or Clean? clean

How many steps should the agent run? 4

Step 1: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Simulation finished after 4 step(s): 0 clean action(s), 4 move(s).
Final location: room A | Final state: {'A': 'Clean', 'B': 'Clean'}

------------------------------------------------------
[1] Two rooms: A and B (Tasks 1-3)
[2] Three rooms: A, B and C with text-based grid (Bonus Task)
[3] Exit
Choose an option (1-3): 3

Program ended.
```

Run 3 observation. Since both rooms were clean from the start, Rule 1 never fired. The agent moved between A and B on all 4 steps and never cleaned anything, which is wasted effort because the two-room agent has no rule for stopping.

Sample Run 4: Bonus Task, Three Rooms

Inputs: option 2, room A clean, rooms B and C dirty, 5 steps (the same inputs as the bonus sample output).

Console Output - Sample Run 4

```
======================================================
RULE-BASED AGENT: VACUUM CLEANER SIMULATION
======================================================
[1] Two rooms: A and B (Tasks 1-3)
[2] Three rooms: A, B and C with text-based grid (Bonus Task)
[3] Exit
Choose an option (1-3): 2

Is room A Dirty or Clean? clean
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
```

```
[A-Agent]  [B-Dirty]  [C-Dirty]
Moving to room B
[A-Clean]  [B-Agent]  [C-Dirty]

Step 2:
[A-Clean]  [B-Agent]  [C-Dirty]
Room B has been cleaned.
[A-Clean]  [B-Agent]  [C-Dirty]

Step 3:
[A-Clean]  [B-Agent]  [C-Dirty]
Moving to room C
[A-Clean]  [B-Clean]  [C-Agent]

Step 4:
[A-Clean]  [B-Clean]  [C-Agent]
Room C has been cleaned.
[A-Clean]  [B-Clean]  [C-Agent]

Step 5:
[A-Clean]  [B-Clean]  [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean]  [B-Clean]  [C-Agent]

Simulation finished after 5 step(s): 2 clean action(s), 2 move(s), 1 idle step(s).
Final location: room C | Final state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

------------------------------------------------------
[1] Two rooms: A and B (Tasks 1-3)
[2] Three rooms: A, B and C with text-based grid (Bonus Task)
[3] Exit
Choose an option (1-3): 3

Program ended.
```

Run 4 observation. The agent randomly picked B first, cleaned it, moved to C, cleaned it and went idle on step 5 once every room was clean. The steps match the bonus sample output line for line; only the menu and the two summary lines at the end are extra.

Sample Run 5: Bonus Task, Same Inputs, Different Random Choice

Inputs: option 2, room A clean, rooms B and C dirty, 5 steps (exactly the same as Run 4).

Console Output - Sample Run 5

```
======================================================
RULE-BASED AGENT: VACUUM CLEANER SIMULATION
======================================================
[1] Two rooms: A and B (Tasks 1-3)
[2] Three rooms: A, B and C with text-based grid (Bonus Task)
[3] Exit
Choose an option (1-3): 2

Is room A Dirty or Clean? clean
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent]  [B-Dirty]  [C-Dirty]
Moving to room C
[A-Clean]  [B-Dirty]  [C-Agent]

Step 2:
[A-Clean]  [B-Dirty]  [C-Agent]
Room C has been cleaned.
[A-Clean]  [B-Dirty]  [C-Agent]
```

```
Step 3:
[A-Clean]  [B-Dirty]  [C-Agent]
Moving to room B
[A-Clean]  [B-Agent]  [C-Clean]

Step 4:
[A-Clean]  [B-Agent]  [C-Clean]
Room B has been cleaned.
[A-Clean]  [B-Agent]  [C-Clean]

Step 5:
[A-Clean]  [B-Agent]  [C-Clean]
All rooms are clean. Agent is idle.
[A-Clean]  [B-Agent]  [C-Clean]

Simulation finished after 5 step(s): 2 clean action(s), 2 move(s), 1 idle step(s).
Final location: room B | Final state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

------------------------------------------------------
[1] Two rooms: A and B (Tasks 1-3)
[2] Three rooms: A, B and C with text-based grid (Bonus Task)
[3] Exit
Choose an option (1-3): 3

Program ended.
```

Run 5 observation. This time the agent randomly picked C first and then B. Both runs still cleaned every room within 4 steps and then went idle, so the random choice changes the order but not the result.

Sample Run 6: Bonus Task, All Three Rooms Dirty

Inputs: option 2, rooms A, B and C dirty, 7 steps.

Console Output - Sample Run 6

```
======================================================
RULE-BASED AGENT: VACUUM CLEANER SIMULATION
======================================================
[1] Two rooms: A and B (Tasks 1-3)
[2] Three rooms: A, B and C with text-based grid (Bonus Task)
[3] Exit
Choose an option (1-3): 2

Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 7

Step 1:
[A-Agent]  [B-Dirty]  [C-Dirty]
Room A has been cleaned.
[A-Agent]  [B-Dirty]  [C-Dirty]

Step 2:
[A-Agent]  [B-Dirty]  [C-Dirty]
Moving to room C
[A-Clean]  [B-Dirty]  [C-Agent]

Step 3:
[A-Clean]  [B-Dirty]  [C-Agent]
Room C has been cleaned.
[A-Clean]  [B-Dirty]  [C-Agent]

Step 4:
[A-Clean]  [B-Dirty]  [C-Agent]
Moving to room B
[A-Clean]  [B-Agent]  [C-Clean]

Step 5:
```

```
[A-Clean]  [B-Agent]  [C-Clean]
Room B has been cleaned.
[A-Clean]  [B-Agent]  [C-Clean]

Step 6:
[A-Clean]  [B-Agent]  [C-Clean]
All rooms are clean. Agent is idle.
[A-Clean]  [B-Agent]  [C-Clean]

Step 7:
[A-Clean]  [B-Agent]  [C-Clean]
All rooms are clean. Agent is idle.
[A-Clean]  [B-Agent]  [C-Clean]

Simulation finished after 7 step(s): 3 clean action(s), 2 move(s), 2 idle step(s).
Final location: room B | Final state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

------------------------------------------------------------
[1] Two rooms: A and B (Tasks 1-3)
[2] Three rooms: A, B and C with text-based grid (Bonus Task)
[3] Exit
Choose an option (1-3): 3

Program ended.
```

Run 6 observation. The agent cleaned A first (Rule 1), then randomly chose C, cleaned it, moved to B and cleaned it. Every room was clean after step 5, so the agent stayed idle for the last two steps instead of wandering around (3 clean actions, 2 moves, 2 idle steps).

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. What rules does the agent follow?

The two-room agent follows two condition-action rules. Rule 1: if the current room is dirty, clean it. Rule 2: if the current room is clean, move to the other room. The bonus agent keeps Rule 1, changes Rule 2 to "move to a randomly chosen dirty room," and adds Rule 3: if every room is clean, stay idle.

2. How does the agent decide when to clean and when to move?

It only looks at the room it is standing in. On each step, perceive_and_act() checks that room with is_dirty() and carries out the first rule that matches, so a dirty room always gets cleaned before the agent goes anywhere, and a clean room always triggers a move. The agent does not plan ahead or remember earlier steps; it reacts to its current percept, which is what makes it a simple rule-based (reflex) agent.

3. What happens if both rooms are already clean at the start?

The agent never cleans anything. Rule 1 never applies, so Rule 2 fires on every step and the agent keeps moving between A and B until the steps run out. Run 3 shows this: 4 steps, 0 clean actions and 4 moves. Those moves are wasted, but the agent can't tell because it has no rule for stopping and never checks the other room. Adding an idle rule, like the bonus version's "All rooms are clean. Agent is idle.", fixes this.

4. How would you modify the agent to handle more than two rooms?

Keep all the rooms in the same dictionary (for example A, B, C and D) and change the move logic, since "the other room" no longer makes sense. The agent can either visit the rooms in a fixed order (A to B to C and back to A) or, like my bonus version, check every room in the dictionary and move to a dirty one, picked at random or by distance. It also needs a rule for when everything is clean so it stops wasting moves. MultiRoomVacuumAgent already works this way: it loops over every room in the environment, so adding a fourth room only means adding "D" to the room list.